



UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE MATEMATICHE, FISICHE E NATURALI

Project report

CS Api Gateway

Cloud Computing Technologies (Carlo Solimando)

Purpose

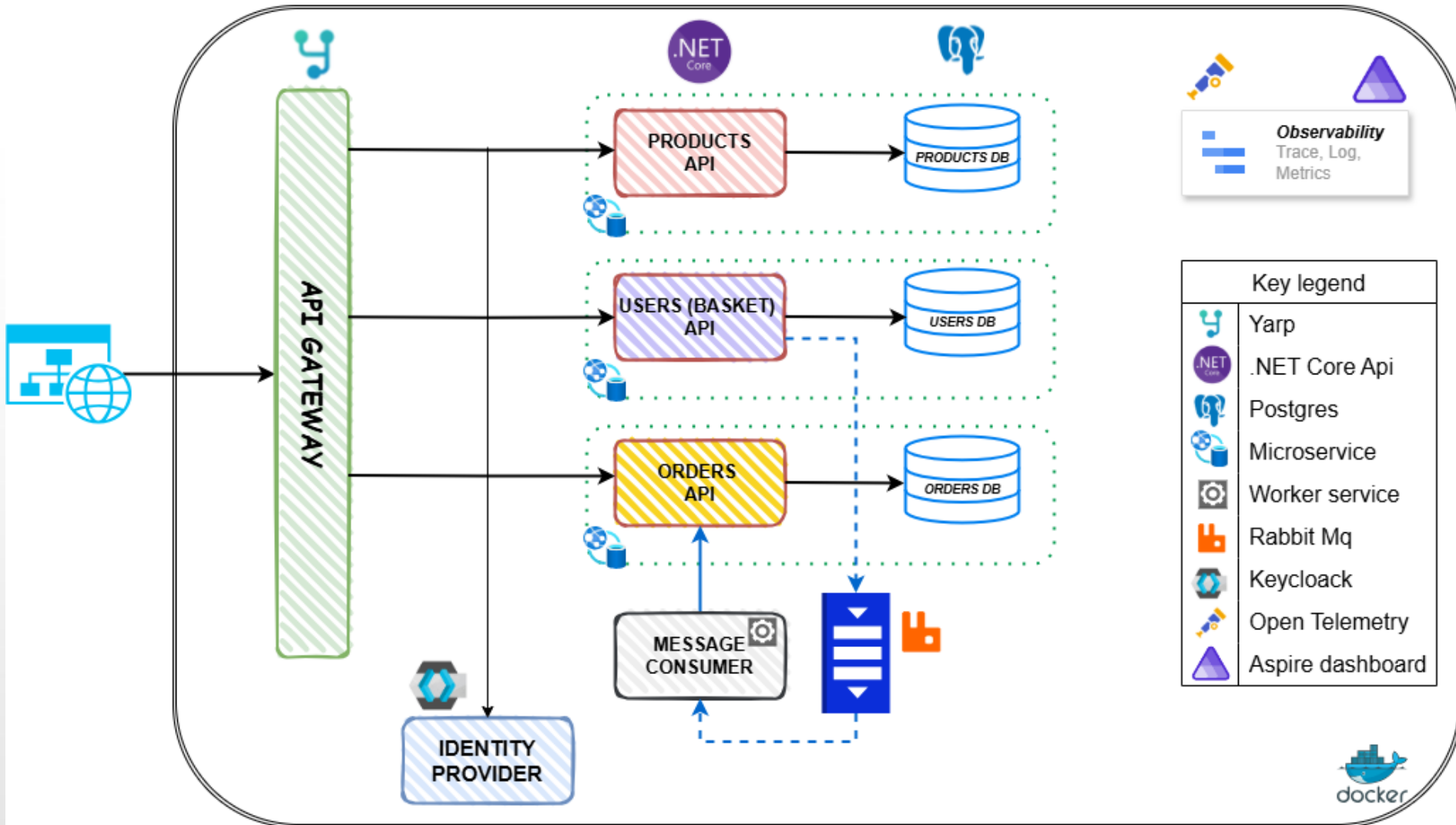
The goal of this presentation is to describe the design and the implementation of a microservices application for product purchasing.

The application was implemented in an API gateway architecture in order to support advanced non-functional features, such as: load balancing, security, fault tolerance and observability.

Agenda

- Reference architecture
- Technology stack
- Functional properties & Implementation details
- Non-functional properties
- Demo use cases
- Potential improvements

Reference architecture



Technology stack

- **.NET Core:** reference framework for the microservices (api) and for the service worker that reads messages in the queue.
- **YARP** (Yet another reverse proxy ☺): is a highly customizable reverse proxy library for .NET and it's designed to provide an easy-to-use proxy framework.
- **Keycloak:** is an open-source identity and access management solution.
- **PostgresSQL:** is a powerful, open-source object-relational database.
- **RabbitMQ:** is a powerful, enterprise grade open-source messaging and streaming broker that enables decoupled communication for applications.
- **OpenTelemetry:** is a collection of APIs, SDKs and tools to instrument, generate, collect, and export telemetry data.
- **Aspire dashboard:** a sophisticated dashboard for comprehensive app monitoring and inspection.
- **Docker:** is a platform designed to build, share, and run container applications.

Functional properties & Implementation details

- **Product catalog management:** The user is given the ability to manage the product catalog thanks to a microservice that exposes a Rest API. The implementation uses the “ASP.NET Core Minimal API” to expose CRUD operations. For data persistence, the “Entity Framework Core” (as ORM) and a Postgres database were used.
- **Users purchase management:** The user is given the ability to purchase products thanks to a microservice that exposes a Rest API. The completion of the purchase process is simulated by sending a message to a queue (RabbitMQ) with the information contained in the shopping cart. The technologies used for the microservice are the same as those described in the previous sections.
- **Order management:** Order management occurs in the background via a .NET Worker service that listens to the queue; whenever a message is published, it processes it and calls the microservice, which exposes a Rest API, which is used to process and persist order information. The technologies used for the microservice are the same as those described in the previous sections.

Non-functional properties

- **Security:** Access management and configuration is handled via the Keycloak identity provider, while microservices security has been implemented using a JWT-bearer authentication.
- **Load balancing:** is managed through YARP configuration by setting the target nodes and the balancing policy. YARP ships with the following built-in policies:
 - *FirstAlphabetical*: select the alphabetically first available destination without considering load.
 - *Random*: select a destination randomly.
 - *PowerOfTwoChoices*: select two random destinations and then select the one with the least assigned requests.
 - *RoundRobin*: select a destination by cycling through them in order (used for the demo).
 - *LeastRequests*: select the destination with the least assigned requests.
- **Fault tolerance:** service interruption management is implemented by leveraging the destination health check functionality provided by YARP, which offers two modes: active and passive (this feature is effective when used in conjunction with load balancing). In active mode (used for the demo), cyclic probing is performed on services either to exclude those that are in unhealthy state or to re-enable traffic to those that have returned to healthy state.
- **Observability:** system observability is ensured through the use of the framework provided by OpenTelemetry, while all tracked information is made available for monitoring via the Aspire dashboard.

Demo use cases

- **Product catalog management:**
 - Explore (and test?) products api definition through Swagger UI (direct service connection)
 - Test products microservice trough postman by calling api endpoint exposed by the API Gateway
 - Use 3 Product microservice instance to show load balancing, rate limitier and fault tolerance (through health check)
- **Users purchase management:**
 - Explore (and test?) users (and basket) api definition through Swagger UI (direct service connection)
 - Test users microservice trough postman by calling api endpoint exposed by the API Gateway
 - Show the security implementation of an api acquiring a bearer token from Keycloak and then use it to call the secured api.
 - Show the creation of a user basket and the related queue message
- **Order management:**
 - Explore (and test?) orders api definition through Swagger UI (direct service connection)
 - Test orders microservice trough postman by calling api endpoint exposed by the API Gateway
 - Show the service worker that read the message from the queue and create ad oder.
- **Miscellaneous:**
 - Open PG admin portal to show microservices database data
 - Open Keycloak to show realm and users configuration
 - Open Rabbit MQ management portal to show queues and messages
 - Open Aspire dashboard to show observability

Potential improvements

- Service registry & discovery: implement service registry & discovery patterns.
- Implements IoC through pipelines
- Improve security implementation
- Explore other load balancing scenarios: for example using docker swarm, Kubernetes or azure container app
- Improve observability: for example exploring Grafana tools
- Explore Aspire framework to speed up and improve development and deployment experience.



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE MATEMATICHE,
FISICHE E NATURALI

Thank you!

Demo:
let's play 😊

Cloud Computing Technologies (Carlo Solimando)

